



Examen: Diseño de sistemas y Base de Datos

Caso 3-A: Logística de Distribución Regional

05/02/26

Hidalgo García Luvia Magali 243732

Montesinos Grajales Alix Anahi 243777

Velazquez Tovilla Mario Alberto 243716

5 B

Universidad Politécnica de Chiapas

2) Restricciones identificadas (UNIQUE, CHECK, NOT NULL, integridad referencial).

1. Restricciones UNIQUE (Unicidad)

Estas reglas garantizan que no existan datos duplicados en el sistema:

- **Nombres de Categorías:** No puede haber dos categorías llamadas igual (ej. dos veces "Lácteos").
- **RFC de Proveedores y Comercios:** El RFC es único; el sistema bloquea intentos de registrar el mismo RFC dos veces.
- **Ficha de Inventario:** Solo puede existir un registro de inventario por cada producto.
- **Productos en el Pedido:** No se permite capturar el mismo producto dos veces en un solo pedido (evita líneas duplicadas).
- **Orden de Ruta:** En una ruta de reparto, no pueden existir dos paradas con el mismo número de orden secuencial.

2. Restricciones CHECK (Validaciones Lógicas)

Estas reglas actúan como filtros automáticos de calidad:

- **Valores Positivos:** Precios, pesos, inventarios, capacidades de carga y kilometrajes deben ser siempre mayores o iguales a cero.
- **Fechas Coherentes:** La fecha de entrega solicitada no puede ser anterior a la fecha del pedido. Los vehículos deben ser del año 2000 en adelante.
- **Límites Financieros:** El saldo deudor de una tienda no puede superar su crédito autorizado. El total del pedido debe ser igual a la suma de Subtotal + IVA.
- **Requisitos de Personal:** Los conductores deben ser mayores de 21 años y su licencia debe estar vigente (fecha de vencimiento futura).
- **Consistencia de Entregas:** La cantidad entregada más la dañada no puede superar lo que se solicitó originalmente.

- **Formato RFC:** El texto del RFC debe cumplir con el patrón oficial (letras y números correctos).
- **Estados Válidos:** Los vehículos y pedidos solo pueden tener estados predefinidos (como "Disponible", "En Ruta", "Entregado"), no se permite texto libre.

3. Restricciones NOT NULL (Datos Obligatorios)

Estas reglas impiden dejar campos vacíos importantes:

- **Identidad:** El Nombre y RFC son obligatorios para todos los proveedores, comercios y productos.
- **Tiempos:** Las fechas de pedido, entrada de almacén, ruta y contratación son obligatorias.
- **Cantidades:** No se permiten registros con cantidad vacía o nula en pedidos ni en movimientos de almacén.
- **Estado:** Los vehículos y conductores siempre deben tener un estado asignado; no pueden estar en "limbo".

4. Restricciones FOREIGN KEY (Integridad Referencial)

Estas reglas protegen las relaciones entre los datos:

- **Protección de Historial (ON DELETE RESTRICT):** No se puede eliminar un proveedor si tiene productos asociados, ni un comercio si tiene pedidos. El sistema protege la historia.
- **Existencia Previa:** No se puede asignar una ruta a un vehículo o conductor que no existe en el catálogo. No se puede vender un producto que no está dado de alta.
- **Limpieza Automática (ON DELETE CASCADE):** Si se elimina un pedido (solo en casos administrativos especiales), el sistema borra automáticamente sus líneas de detalle para no dejar datos huérfanos.

3) Queries SQL de ejemplo: SELECT con JOINs, agregaciones, GROUP BY, HAVING, CASE/COALESCE.

1. Pedidos confirmados sin ruta (LEFT JOIN)

Objetivo: Detectar pedidos listos que se quedaron "olvidados" sin asignar.

Explicación: Usamos LEFT JOIN para encontrar pedidos que no tienen correspondencia en la tabla de rutas (ID_Ruta IS NULL).

```
SELECT p.ID_Pedido, c.Nombre_Comercio, p.Total
FROM Pedidos p
JOIN Comercios c ON p.ID_Comercio = c.ID_Comercio
LEFT JOIN Ruta_Pedido rp ON p.ID_Pedido = rp.ID_Pedido
WHERE p.Estado = 'Confirmado' AND rp.ID_Ruta IS NULL;
```

2. Alerta de Sobrecarga (Agregación + CASE)

Objetivo: Verificar si el peso de la carga supera la capacidad del camión.

Explicación: Sumamos el peso de los productos y usamos CASE para etiquetar visualmente si hay "Peligro" o si es "Seguro".

SQL

```
SELECT v.Placas, v.Capacidad_Kg,
       SUM(dp.Cantidad_Solicitada * prod.Peso_Unitario_Kg) AS Carga_Total,
       CASE WHEN SUM(dp.Cantidad_Solicitada * prod.Peso_Unitario_Kg) > v.Capacidad_Kg
            THEN 'PELIGRO' ELSE 'OK' END AS Estado
FROM Rutas r
JOIN Vehiculos v ON r.Placas_Vehiculo = v.Placas
JOIN Ruta_Pedido rp ON r.ID_Ruta = rp.ID_Ruta
JOIN Pedidos p ON rp.ID_Pedido = p.ID_Pedido
JOIN Detalle_Pedido dp ON p.ID_Pedido = dp.ID_Pedido
JOIN Productos prod ON dp.ID_Producto = prod.ID_Producto
WHERE r.Estado = 'Planificada'
GROUP BY r.ID_Ruta, v.Placas, v.Capacidad_Kg;
```

3. Rutas con Incidencias (GROUP BY + HAVING)

Objetivo: Filtrar solo las rutas completadas que tuvieron problemas.

Explicación: El HAVING es el filtro clave aquí; se ejecuta después del GROUP BY para descartar todas las rutas perfectas (con 0 incidencias).

```
SELECT r.ID_Ruta, c.Nombre_Completo,
       SUM(CASE WHEN p.Estado =
'Entregado con incidencia' THEN 1 ELSE
0 END) AS Incidencias
FROM Rutas r
JOIN Conductores c ON
r.Licencia_Conductor =
c.Numero_Licencia
JOIN Ruta_Pedido rp ON r.ID_Ruta =
rp.ID_Ruta
JOIN Pedidos p ON rp.ID_Pedido =
p.ID_Pedido
WHERE r.Estado = 'Completada'
GROUP BY r.ID_Ruta, c.Nombre_Completo
HAVING SUM(CASE WHEN p.Estado =
'Entregado con incidencia' THEN 1 ELSE
0 END) > 0;
```

4. Gasto por Proveedor (COALESCE)

Objetivo: Calcular el gasto total evitando errores con valores nulos.

Explicación: COALESCE convierte los NULL en 0. Esto asegura que si un proveedor no tiene movimientos, el reporte muestre \$0.00 en lugar de un campo vacío.

```
SELECT prov.Nombre_Empresa,
       COALESCE(SUM(mi.Cantidad *
mi.Costo_Unitario), 0) AS Gasto_Total
FROM Proveedores prov
LEFT JOIN Movimientos_Inventario mi ON
prov.ID_Proveedor = mi.ID_Proveedor
AND mi.Tipo_Movimiento = 'ENTRADA'
GROUP BY prov.Nombre_Empresa
ORDER BY Gasto_Total DESC;
```

5. Valor del Inventario (Agregación Simple)

Objetivo: Valorar cuánto dinero hay almacenado por categoría.

Explicación: Multiplica stock por precio base y agrupa el resultado, fundamental para la contabilidad básica.

```
SELECT cat.Nombre,  
       SUM(inv.Cantidad_Disponible *  
prod.Precio_Base) AS Valor_Total  
FROM Inventario inv  
JOIN Productos prod ON inv.ID_Producto  
= prod.ID_Producto  
JOIN Categorías cat ON  
prod.ID_Categoría = cat.ID_Categoría  
GROUP BY cat.Nombre;
```

4) Reportes implementados cómo VIEWS.

View 1:V_Inventario_Critico

Inventario crítico: productos cuyo stock actual no alcanza para cubrir ni 15 días de ventas, debe funcionar incluso para productos sin ventas recientes ademas de manejar el caso de 'cero ventas recientes'.

Justificación: Soluciona el problema matemático de la "división por cero" cuando un producto no tiene ventas. En lugar de dar error, el sistema asigna cobertura infinita y lo etiqueta como "Estancado", permitiendo distinguir entre desabasto crítico y mercancía inmovilizada

```
CREATE OR REPLACE VIEW V_Inventario_Critico AS
WITH Ventas_Diarias AS (
    SELECT
        dp.ID_Producto,
        COALESCE(SUM(dp.Cantidad_Solicitada), 0) / 30.0 AS Promedio_Venta
    FROM Detalle_Pedido dp
    JOIN Pedidos p ON dp.ID_Pedido = p.ID_Pedido
    WHERE p.Fecha_Pedido::DATE >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY dp.ID_Producto
)
SELECT
    prod.Nombre AS Producto,
    cat.Nombre AS Categoria,
    inv.Cantidad_Disponible AS Stock_Actual,
    ROUND(COALESCE(vd.Promedio_Venta, 0), 2) AS Ventas_Diarias_Prom,
    CASE
        WHEN COALESCE(vd.Promedio_Venta, 0) = 0 THEN 9999
        ELSE ROUND(inv.Cantidad_Disponible / vd.Promedio_Venta, 1)
    END AS Dias_Cobertura,
    CASE
        WHEN COALESCE(vd.Promedio_Venta, 0) = 0 THEN 'Estancado (Sin ventas recientes)'
        WHEN (inv.Cantidad_Disponible / vd.Promedio_Venta) < 15 THEN 'CRÍTICO (<15 días)'
        ELSE 'Suficiente'
    END AS Estado_Inventario
FROM Productos prod
JOIN Inventario inv ON prod.ID_Producto = inv.ID_Producto
JOIN Categorías cat ON prod.ID_Categoria = cat.ID_Categoria
LEFT JOIN Ventas_Diarias vd ON prod.ID_Producto = vd.ID_Producto
WHERE
    COALESCE(vd.Promedio_Venta, 0) = 0
    OR (inv.Cantidad_Disponible / vd.Promedio_Venta) < 15;
```

View 2:V_Resumen_Zona

Resumen de pedidos por zona geográfica: total de pedidos, entregas exitosas, entregas con incidencia, y tasa de éxito, esto es solo zonas con actividad significativa."

Justificación: Implementa contadores condicionales para calcular la tasa de éxito real. El requisito de "actividad significativa" lo resolvimos con un filtro de agrupación (HAVING), eliminando zonas con un volumen irrelevante para enfocar el análisis en regiones con impacto comercial real.

```
CREATE OR REPLACE VIEW V_Resumen_Zona AS
SELECT
  c.Zona,
  COUNT(p.ID_Pedido) AS Total_Pedidos,

  SUM(CASE WHEN p.Estado = 'Entregado' THEN 1 ELSE 0 END) AS Entregas_Exitosas,
  SUM(CASE WHEN p.Estado = 'Entregado con incidencia' THEN 1 ELSE 0 END) AS Con_Incidencias,

  ROUND(
    (SUM(CASE WHEN p.Estado = 'Entregado' THEN 1 ELSE 0 END)::DECIMAL /
     NULLIF(COUNT(p.ID_Pedido), 0)) * 100,
    1) AS Tasa_Exito_Porcentaje
FROM Pedidos p
JOIN Comercios c ON p.ID_Comercio = c.ID_Comercio
WHERE p.Estado IN ('Entregado', 'Entregado con incidencia')
GROUP BY c.Zona
HAVING COUNT(p.ID_Pedido) > 5;
```

View 3:V_Productividad_Conductores

"Productividad de conductores: rutas asignadas, pedidos entregados, promedio por ruta, y una clasificación de rendimiento (el equipo define los rangos y los justifica).

Justificación: Transforma datos operativos en métricas de Recursos Humanos. Definimos el rango "Alto" en más de 40 entregas mensuales (aprox. 2 diarias) para eliminar la

subjetividad en la evaluación, clasificando automáticamente el desempeño en base a eficiencia pura.

```
CREATE OR REPLACE VIEW V_Productividad_Conductores AS
SELECT
    cond.Nombre_Completo AS Conductor,
    COUNT(DISTINCT r.ID_Ruta) AS Rutas_Asignadas,
    COUNT(DISTINCT rp.ID_Pedido) AS Total_Pedidos_Entregados,

    ROUND(COUNT(DISTINCT rp.ID_Pedido)::DECIMAL / NULLIF(COUNT(DISTINCT r.ID_Ruta), 0), 1) AS
    Promedio_Pedidos_Ruta,

    CASE
        WHEN COUNT(DISTINCT rp.ID_Pedido) > 40 THEN 'Alto Rendimiento (Estrella)'
        WHEN COUNT(DISTINCT rp.ID_Pedido) BETWEEN 15 AND 40 THEN 'Promedio'
        ELSE 'Bajo / En Capacitación'
    END AS Clasificacion
FROM Conductores cond
JOIN Rutas r ON cond.Numero_Licencia = r.Licencia_Conductor
JOIN Ruta_Pedido rp ON r.ID_Ruta = rp.ID_Ruta
JOIN Pedidos p ON rp.ID_Pedido = p.ID_Pedido
WHERE p.Estado IN ('Entregado', 'Entregado con incidencia')
GROUP BY cond.Numero_Licencia, cond.Nombre_Completo;
```

View 4:V_Utilizacion_Flota

Utilización de flota: por vehículo, cuántas rutas hizo en el mes y si está subutilizado, en punto óptimo, o sobrecargado (el equipo define umbrales).

Justificación: Usa una conexión inclusiva (LEFT JOIN) para visibilizar vehículos "Subutilizados" que tienen cero rutas. Establece el umbral de "Sobrecarga" en más de 20 rutas al mes para alertar sobre el desgaste excesivo de activos y prevenir averías mecánicas.

```
CREATE OR REPLACE VIEW V_Utilizacion_Flota AS
SELECT
    v.Placas,
    v.Modelo,
    v.Capacidad_Kg,
    COUNT(r.ID_Ruta) AS Rutas_Mes_Actual,

    CASE
        WHEN COUNT(r.ID_Ruta) = 0 THEN 'Subutilizado (Ocioso)'
        WHEN COUNT(r.ID_Ruta) BETWEEN 1 AND 20 THEN 'Punto
Óptimo' ELSE 'Sobrecargado (>20 rutas)'
    END AS Estado_Uso
FROM Vehiculos v
LEFT JOIN Rutas r ON v.Placas = r.Placas_Vehiculo
    AND r.Fecha_Ruta::DATE >= DATE_TRUNC('month', CURRENT_DATE)
GROUP BY v.Placas, v.Modelo, v.Capacidad_Kg;
```

View 5: V_Rentabilidad_Proveedor

Rentabilidad por proveedor: comparar lo que se gastó en comprar sus productos vs lo que se obtuvo al venderlos. Mostrar margen.

Justificación: Cruza financieramente las entradas de almacén (costo) contra las salidas por pedido (venta), esto es para el margen de utilidad real, identificando proveedores que generan ganancia neta para la empresa en lugar de solo volumen de facturación.

```
CREATE OR REPLACE VIEW V_Rentabilidad_Proveedor AS
SELECT
    prov.Nombre_Empresa,

    COALESCE(SUM(mi.Cantidad * mi.Costo_Unitario), 0) AS Total_Gastado_Compras,

    (
        SELECT COALESCE(SUM(dp.Subtotal), 0)
        FROM Detalle_Pedido dp
        JOIN Productos prod ON dp.ID_Producto = prod.ID_Producto
        JOIN Proveedor_Producto pp ON prod.ID_Producto = pp.ID_Producto
        WHERE pp.ID_Proveedor = prov.ID_Proveedor AND pp.Es_Proveedor_Principal = TRUE
    ) AS Total_Ventas_Estimadas,

    (
        (SELECT COALESCE(SUM(dp.Subtotal), 0)
        FROM Detalle_Pedido dp
        JOIN Productos prod ON dp.ID_Producto = prod.ID_Producto
        JOIN Proveedor_Producto pp ON prod.ID_Producto = pp.ID_Producto
        WHERE pp.ID_Proveedor = prov.ID_Proveedor AND pp.Es_Proveedor_Principal = TRUE)
        -
        COALESCE(SUM(mi.Cantidad * mi.Costo_Unitario), 0)
    ) AS Margen_Bruto
FROM Proveedores prov
LEFT JOIN Movimientos_Inventario mi ON prov.ID_Proveedor = mi.ID_Proveedor
    AND mi.Tipo_Movimiento = 'ENTRADA'
GROUP BY prov.ID_Proveedor, prov.Nombre_Empresa;
```

5) Respuestas escritas a las preguntas de defensa de su caso.

¿Por qué escogieron ese caso?

Elegimos el siguiente caso referente que analizando los 2 casos más, nos dimos cuenta que contaba con un nivel menos complejo para que resolverlo y analizar en la base de datos las tablas y no tener problema al momento de crear el mapa de entidad relacion y tambien las restricciones para identificarlas concretamente y que además de que en este caso era mucho más fácil visualizar cómo fluyen los datos para crear la queries correctamente en SQL.

¿Por qué lo diseñaron así?

Estuvimos analizando sobre el diseño y creamos la tablas a base de los datos que se nos presentaron en la problemática, cada tabla tiene atributos y puntos claves que nos menciona en el problema, abarcando desde este punto cada tabla tiene información relacionada con la problemática para resolver y obtener los reportes y consultas que se mencionan en el problema, primero identificamos cada posible relación y las posibles tablas según lo que leíamos del escenario, pasamos de 15 tablas a 13, analizamos primero lo de cuando el proveedor envía al almacén y se registra la entrada, luego cuando el proveedor envía a la tienda y se verifica el stock suficiente para poder confirmar el pedido, luego se crea la ruta y se la asigna al vehículo y al conductor para la ruta, al ir desglosando cada parte como eso sacamos las entidades que las convertimos en tablas